

Future Balance Lab & Production AI Considerations

Overflow material intentionally deferred from the first two implementation phases

Project: Dice and Destiny v2

Prepared: August 1, 2026

Status: Reference document; not authorized implementation scope

Companion document: Local Two-Controller Battle & Machine Learning Mirror-Match Pilot

Purpose: Preserve valuable future directions without allowing them to expand or delay the immediate two-phase Blade Warden mirror-match pilot.

1. How to use this document

This document contains ideas discussed during the architecture evaluation that are useful after the first local two-controller battle and first learning pilot succeed. Nothing here is required for approval or completion of those two phases. Each topic includes a suggested trigger for reconsideration so future work can be pulled forward deliberately rather than by accident.

2. Generalized policy for a large creature roster

Training and maintaining one independent model per creature will not scale well to hundreds of creatures. A future policy should be conditioned on structured content mechanics rather than memorizing only content IDs. Relevant features include card cost and operations, dice faces and symbols, ability qualifications and effects, status triggers, resource state, health/deck structure, elements, and defensive profiles.

A universal policy can share experience across creatures and evaluate newly authored content sooner. Specialist policies can remain useful for unusual mechanics or for training a best response against an outlier.

Revisit when: At least three mechanically contrasting completed creatures exist and the Blade Warden pilot demonstrates stable observation and action schemas.

3. Population self-play and best responses

Two current models repeatedly fighting only each other can converge to shared blind spots. A stronger future training population should sample opponents from current policies, older checkpoints, heuristic agents, intended easy/normal/hard PvE policies, search agents, and creature-specific best responses.

Periodically train an adversarial best-response policy whose only task is to exploit the current champion. A large best-response advantage indicates that the champion is not robust even if its self-play win rate appears stable.

Revisit when: The first policy reliably beats random play, is competitive with the heuristic baseline, and produces reproducible checkpoints across multiple seeds.

4. Search agents and expert distillation

Monte Carlo search or MCTS may become a stronger development-time expert than early reinforcement learning because battle states are cloneable and stochastic futures can be sampled. Hidden information would require sampling states consistent with each actor's knowledge rather than searching omnisciently.

Runtime search can be expensive on low-end machines. A future workflow can use search to generate expert demonstrations and then distill that behavior into a small policy. The shipped game would run the cheap policy while development retains the expensive search oracle.

Revisit when: The legal action enumerator and simulator can branch from cloned battle states without changing authoritative results.

5. Defining balance beyond a mirror match

A 50 percent Blade Warden mirror result demonstrates seat symmetry under the evaluated policies; it does not prove that Blade Warden is appropriately strong against other creatures. A healthy roster may include intentional counters and strategic cycles rather than forcing every pair toward 50 percent.

Future evidence	Question answered
Mirror matches	Are the seats, priority rules, and controllers symmetric?
Cross-matchup matrix	Which creatures have favorable and unfavorable relationships?
Policy-strength ladder	Does the conclusion persist from random through heuristic and expert play?
Best-response exploitability	Can a targeted opponent discover a large strategic weakness?
Human playtests	Are the strategies discoverable, readable, and enjoyable for real players?
Pace and variance metrics	Are outcomes too slow, too volatile, or dominated by nonproductive turns?

Revisit when: At least three completed creatures and a stable expert or near-expert policy pool exist.

6. Statistical scale and adaptive sampling

At a true 50 percent win rate, approximately 1,000 independent games provide a worst-case 95 percent sampling margin near ± 3.1 percentage points; 10,000 games reduce that margin to roughly ± 1 point. Matched random seeds before and after a content change can reduce noise further.

A 300-creature roster contains 44,850 unordered pairings before accounting for levels, builds, seat order, or policy strength. Exhaustive fixed-size evaluation will become wasteful. Future tooling should prioritize benchmark opponent suites, rating or strength models, uncertainty-aware sampling, and additional games only near decision boundaries.

Revisit when: The experiment catalog expands beyond a small hand-curated matchup set or evaluation time becomes a material development bottleneck.

7. Production enemy AI

A trained reinforcement-learning policy can eventually replace or complement the D100 controller in the shipped local game. Training is computationally expensive; inference from a small policy is usually cheap. The model should receive a viewer-safe observation and a list of legal action candidates, then return one candidate index. The authority remains responsible for command validation and state mutation.

Production concern	Future approach
Runtime cost	Use a small MLP or candidate-scoring network; infer only when a decision is required.
Packaging	Export to ONNX or a compact custom weight format; consider pure-Go inference for a fixed small architecture.
Failure handling	Retain the D100 or heuristic controller as a deterministic fallback.
Versioning	Pin model, observation schema, action schema, engine revision, and content version.
Replay	Record the selected command; do not require cross-platform floating-point inference to reproduce a replay.
Low-end support	Benchmark CPU inference on the minimum target machine and enforce a decision timeout.

Revisit when: A learned policy clearly exceeds the intended PvE baseline and its behavior is suitable for players, not merely for balance analysis.

8. Difficulty levels and personality

The most optimal bot is not automatically the most enjoyable bot. Future production difficulty may use different checkpoints, controlled top-action sampling, deliberate mistake rates, restricted history, or a small search budget. Creature personalities can influence preferences without bypassing legal actions or authority rules.

Illustrative tier	Possible controller
Easy	D100 policy or learned policy with deliberate mistakes
Normal	Learned policy with controlled stochasticity
Hard	Strong learned checkpoint
Expert	Best policy checkpoint
Challenge	Best policy plus a bounded search budget

Revisit when: The policy is being evaluated as a player-facing opponent rather than solely as a development tool.

9. Hardware and compute scaling

The M3 Max is sufficient for the first simulation, search, and reinforcement-learning pilots. CPU throughput is particularly important because the Go engine produces experience. Apple's MPS backend can be benchmarked if neural-network updates become the bottleneck, but CPU-first development is appropriate for a small PPO policy.

Cloud hardware becomes useful only when large hyperparameter sweeps, hundreds of millions of decisions, large self-play populations, or a broad roster evaluation need to be compressed into a shorter schedule. Occasional rental is likely more economical than buying specialized hardware before measured demand exists.

Revisit when: Profiling demonstrates that local training or evaluation time—not environment correctness or experiment design—is the limiting factor.

10. Future game mechanics that change the learning problem

The creature-system working design introduces active teams, deployment, switching, persistent per-creature battle state, elements, level and evolution milestones, and potentially separate defensive profiles. These features enlarge both observations and actions and may invalidate an early fixed-slot policy. They should be added only after the first pilot establishes versioned environment contracts.

- **Teams and switching.** Add active/bench state, switch costs, hand behavior on switch, and team-level terminal conditions.
- **Elements.** Encode matchup modifiers once the design chooses the single layer they affect.
- **Progression.** Evaluate milestone snapshots rather than assuming a final-level model applies to every level.
- **New content operations.** Extend action and observation encoders by mechanic, not by hardcoded creature ID.

Revisit when: Each mechanic is settled enough to have deterministic server rules and focused authority tests.

11. Actual network multiplayer

The local two-controller architecture is a useful foundation for eventual online PvP, but networking is a separate product initiative. It would require identity, session lifecycle, matchmaking or invitations,

transport, reconnect and timeout policy, anti-cheat boundaries, version compatibility, and player-facing UX. None of that is necessary for local model self-play.

Revisit when: There is an explicit player-facing online PvP goal independent of machine-learning balance work.

12. Tooling references for future evaluation

[Gymnasium custom environments](#) — Standard reset/step environment design.

[Gymnasium vector environments](#) — Parallel independent simulation environments.

[Stable Baselines3 Maskable PPO](#) — Invalid-action masking for discrete policy training.

[PettingZoo action masking](#) — Future multi-agent environment conventions.

[Apple accelerated PyTorch](#) — Optional MPS acceleration on Apple Silicon.

[ONNX Runtime C API](#) — Cross-platform production inference option.

[ONNX Runtime quantization](#) — Optional model-size and inference optimization.

13. Deferred decision register

Decision	Reason to defer	Evidence needed
Universal versus specialist policy	One-combatant pilot cannot test generalization	Three or more contrasting combatants
MCTS at runtime	Runtime budget and low-end targets are unknown	Search benchmark and production latency budget
ONNX versus pure Go inference	Actual model architecture is not yet known	Exported policy and packaging benchmark
Cloud compute	No demonstrated local bottleneck	Measured training/evaluation wall time
Player-facing difficulty tiers	Optimal balance policy may not be enjoyable	Human playtests against stable policies
Network PvP	Unrelated to local self-play objective	Separate product decision and requirements